Padrões GRASP – Interface e Polimorfismo

Sabrina de F. Souto.

Um objeto é composto por?

- Características
  - Atributos
- Responsabilidades
  - Métodos

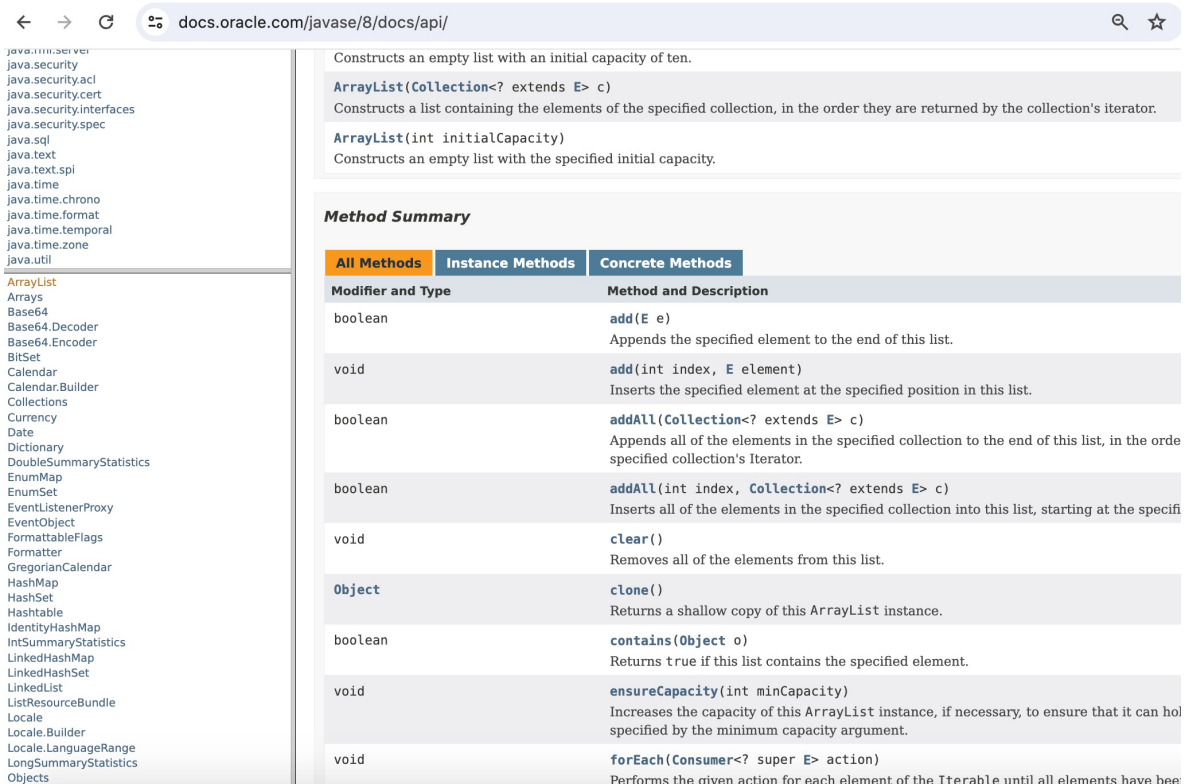
O que é uma assinatura?

- Assinatura consite em:
  - Tipo de Acesso
  - Retorno
  - Nome da Operação
  - Parâmetros

- acesso retorno método(parâmetros)
- *public* ***boolean equals*** *(****Object*** *obj)*
- *Public* ***void setName****(****String*** *novoNome)*

O que é Interface

- O conjunto de todas as assinaturas de um objeto define sua **INTERFACE**
- A Interface de um objeto conhece todas as solicitações que o mesmo pode receber



O que é Tipo?

- Nome usado para denotar uma interface específica
  - Objeto é **do tipo** String
  - Aceita todas as solicitações para as operações definidas na interface String
    - compareToIgnoreCase(String str)
    - concat(String str)
    - endsWith(String suffix)
    - length()
    - ...

Tipo

- Um objeto pode ser de muitos tipos
  - Classe: File
    - Interfaces: Serializable, Comparable
- Objetos muito diferentes podem compartilhar o mesmo tipo
  - Classes: EnumMap e HashMap
    - Interface: Serializable, Cloneable, Map

Sistemas OO

- Objetos são conhecidos apenas pelas suas interfaces
  - Não tem como **pedir** ou **saber** algo sem conhecer a interface

Tipo x Implementação

- A interface de um objeto nada diz sobre sua implementação
- Dois objetos com mesma interface (mesmo Tipo) podem ter implementações diferentes

Assinatura x Implementação

```
public interface Comparable {
    public int compareTo (Comparable c);
}
```

```
public class Funcionario implements Comparable {
    private double salario;
    public int compareTo (Comparable c) {
      if (this.funcao == c.getFuncao()){ ...
      if (this.salário > c.getSalário()){  ...
}
```

```
public class Livro implements Comparable {
    private String isbn;
    private String titulo;
    public int compareTo (Comparable c) {
      if (this.isbn > c.getISBN()){ ...
      if (this.titulo == c.getTitulo ()){ ...
}
```

Qual o ganho?

- Qual o ganho de ter a implementação desvinculada da assinatura, da interface????
  - **Dynamic Binding**

O que é Dynamic Binding?

- A associação entre a assinatura invocada e sua implementação só ocorre em tempo de execução.
  - O programa espera um objeto que implementa uma determinada interface (é de certo Tipo), independente de qual seja esse objeto (Implementação).

E qual é a vantagem disso?

- **Polimorfismo!!!**

- Você pode substituir um objeto por outro, desde que ambos implementes a mesma interface (comportamento desejado), sem prejuízo
  - Eu quero comparar dois objetos quaisquer
    - cadeiras, pessoas, figuras geométricas....
  - Eu só preciso saber se são comparaveis
    - ***implements comparable***

Polimorfismo

- O cliente não precisa saber **como** as cadeiras, as pessoas ou as figuras geométricas são comparadas

- O cliente só sabe que esses objetos **são comparáveis**.
  - Comparable x = new Cadeira();
  - Comparable y = new Cadeira();

  ou
  - Comparable x = new Pessoa();
  - Comparable y = new Pessoa();

  - x.compareTo(y); **ainda é válido!**

Polimorfismo

- Simplificar as definições dos clientes
- Desacoplar objetos entre si
- Variar os inter-relacionamentos em tempo de execução
  - Não sabe como as comparações são feitas
  - Não sabe se compara dois animais ou dois livros (objetos)

Herança

- Classe (*extends*)
  - Herdamos a implementação
  - Mecanismo para compartilhar código
- Interface (subtipo)
  - Quando um objeto pode ser usado no lugar de outro
- Herança de Classe → Herança de Interface

“program to an interface, not an implementation”

- Herança → Reuso de uma família de objetos com a mesma interface
  - Herança de Classe não possibilita polimorfismo
- Herança de Interface permite polimorfismo
  - A flexibilidade vem da possibilidade de mudar a implementação da interface, até em tempo de execução, “*dinamyc binding*"

“program to an interface, not an implementation”

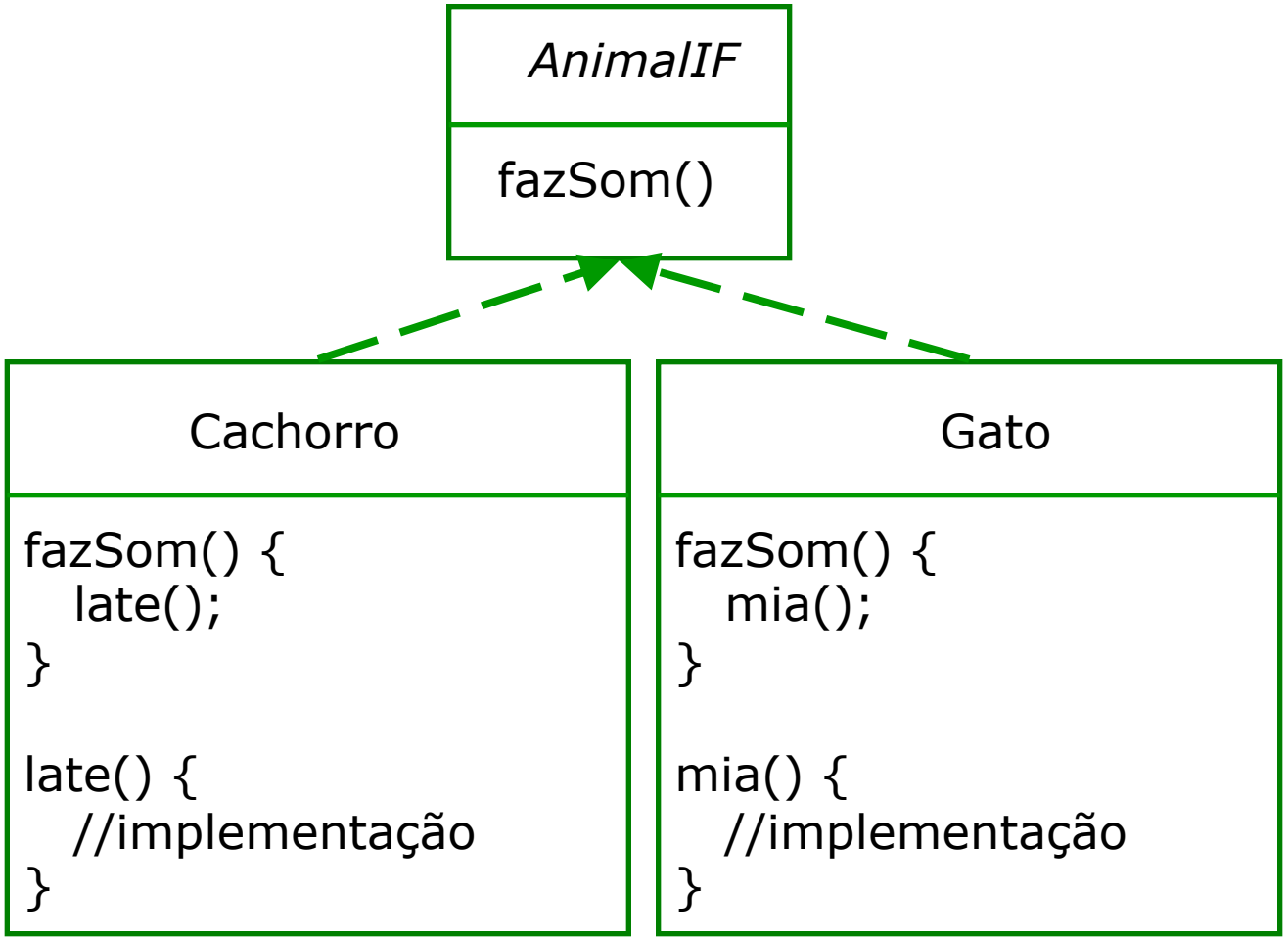
- **Programando para implementação**

```
Cachorro cao = new Cachorro();
cao.late();
```

- **Programando para interface (supertipo)**

```
AnimalIF animal = new Cachorro();
animal.fazSom();
```

“program to an interface, not an implementation”



O que desejamos?

- Obter Polimorfismo a partir de **Herança de Interface**
  - Definir boas Interfaces (Tipos)
    - Identificando o que há de comum entre as várias situações (os vários objetos usados pelos clientes)

Exemplo 01

- Temos três coleções
  - ColeçãoDeAlunos, ColeçãoDeProfessores, ColeçãoDeDisciplinas
- Temos um cliente comum dessas coleções
  - Quero listar todos os alunos com nome "João" e exibí-los numa list box para escolha pelo usuário
  - Idem para listar professores com nome "Alfredo"
  - Idem para listar disciplinas com nome "Programação"

Exemplop 01

- O que fazer?
  - Definir uma interface comum às três coleções
- Todas as coleções tem um comportamento comum
  - Todas são **selecionáveis por nome**
- Os objetos manipulados devem ser nomeáveis
  - Duas boas dicas para interface

Exemplo 01

```
public interface SelecionavelPorNome{
    public Iterator getIteradorPorNome(String nome);
}


public interface Nomeavel {
    public String getNome();
}
```

Exemplo 01

```
public class ColAlunos implements SelecionavelPorNome {
   //...
    Iterator getIteradorPorNome(String nome) {
     // implementação própria
    }
}

public class Aluno implements Nomeavel {
    // ...
    public String getNome() {
     //implementação prórpia
    }
}
```

Exemplo 01

```
public class ColProfessores implements SelecionavelPorNome {
   //...
     Iterator getIteradorPorNome(String nome) {
       // implementação própria
     }
}

public class Professor implements Nomeavel {
     // ...
     public String getNome() {
       //implementação prórpia
     }
}
```

Exemplo 01

```
public class ColDisciplinas implements SelecionavelPorNome {
   //...
     Iterator getIteradorPorNome(String nome) {
       // implementação própria
     }
}

public class Disciplina implements Nomeavel {
     // ...
     public String getNome() {
       //implementação prórpia
     }
}
```

Exemplo 01

CLASSE CLIENTE

```
public class ComponenteDeSelecao {
    Iterator it;

    public ComponenteDeSelecao(SelecionavelPorNome colecao, String nome) {
      it = coleção.getIteradorPorNome(nome);
    }

    public void geraLista() {
      while(it.hasNext()) {
            Nomeavel obj = (Nomeavel) it.next();
      System.out.prinln(obj.getNome());
       }
    }
 }
```

Exemplo 02

- Imagine a seguinte estrutura de classes:
  - Pessoa
  - Atleta *extends* Pessoa
  - Nadador *extends* Atleta
  - Corredor *extends* Atleta
  - Ciclista *extends* Atleta

- Algum problema?

Exemplo 02

- E se tivermos um tipo especial de Atleta, um Triatleta?
  - Pessoa
  - Atleta *extends* Pessoa
  - Nadador *extends* Atleta
  - Corredor *extends* Atleta
  - Ciclista *extends* Atleta
  - **Triatleta *extends* Nadador, Corredor, Ciclista**

- Java não permite herança múltipla!!!

Exemplo 02

- Uma solução usando interface
  - Interfaces
    - Atleta
    - Nadador *implements* Atleta
    - Corredor *implements* Atleta
    - Ciclista *implements* Atleta
  - Classes
    - Pessoa
    - Triatleta *extends* Pessoa *implements* Nadador, Corredor, Ciclista

Exemplo 02

```
public interface Atleta {
   public void aquecer();
}
public interface Nadador implements Atleta {
   public void nadar();
}
public interface Corredor  implements  Atleta {
   public void correr();
}
public interface Ciclista  implements  Atleta {
   public void pedalar();
}
```

Exemplo 02

```
public class Pessoa {
    private String nome;
    private String endereco;

    public Pessoa(String nome) {
        this.setNome(nome);
    }

    //métodos set e get
}
```

Exemplo 02

```
public class Triatleta extends Pessoa implements Nadador, Corredor, Ciclista {
        public Triatleta(String nome) {
       super(nome);
   }
        public String aquecer() {  //Atleta
       return this.nome() + " está aquecendo";
   }
        public  String  nadar() {  //Nadador
        return this.nome() + " está nadando";
   }
        public  String  correr() {  //Corredor
        return this.nome() + " está correndo";
   }
        public void pedalar() {  //Ciclista
        return this.nome() + " está pedalando";
   }
}
```

Como achar interfaces?

→ Procure assinaturas repetidas

- várias classes que representam coisas que podem ser vendidas: **VendávelIF**

→ Procure métodos que poderiam ser usados em aplicações semelhantes

- O cliente chama a interface (reuso e flexibilidade)
- muitas coisas poderiam ser **reserváveis**, não só passagens de avião
- muitas coisas pode ser **alugavéis**, não só fitas de vídeo

→ Procure mudanças futuras (novos objetos que poderiam aparecer) e coloque as semelhanças sob controle de interfaces

Dúvida?